Entrega 4 - Proyecto Integrador

En este Notebook se desarrolla la cuarta entrega del Proyecto Integrador de Ciencia de Datos.

Importación de y preparaciones iniciales

Primero importamos todas las librerías necesarias y descargamos el archivo de datos inicial, el cual contiene más de 75.000 peliculas, pero figuran 148000 porque las peliculas que tienen mas de un genero en vez de aparecer una vez con todos los generos , aparece cada vez con el genero individual.

```
from sklearn.neighbors import NearestNeighbors
import numpy as np
import pandas as pd
import altair as alt
import pandas as pd
from IPython.display import display
```

```
!pip install "vegafusion[embed]>=1.5.0" vegafusion
```

```
Collecting vegafusion>=1.5.0 (from vegafusion[embed]>=1.5.0)
  Downloading vegafusion-2.0.3-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.1 kB)
Collecting arro3-core (from vegafusion>=1.5.0->vegafusion[embed]>=1.5.0)
  Downloading arro3_core-0.8.0-cp311-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (515 bytes)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from vegafusion>=1.5.0->vegafusion[embe
Requirement already satisfied: narwhals>=1.42 in /usr/local/lib/python3.12/dist-packages (from vegafusion>=1.5.0->vegafusior
Downloading vegafusion-2.0.3-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (23.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.8/23.8 MB 35.8 MB/s eta 0:00:00
Downloading arro3_core-0.8.0-cp311-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.0/3.0 MB 27.7 MB/s eta 0:00:00
Installing collected packages: arro3-core, vegafusion
Successfully installed arro3-core-0.8.0 vegafusion-2.0.3
```

```
!pip install "vl-convert-python>=1.6.0"
```

```
Collecting vl-convert-python>=1.6.0
  Downloading vl_convert_python-1.9.0.post1-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.1 kB)
Downloading vl_convert_python-1.9.0.post1-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (33.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 33.5/33.5 MB 16.8 MB/s eta 0:00:00
Installing collected packages: vl-convert-python
Successfully installed vl-convert-python-1.9.0.post1
```

```
alt.data_transformers.enable("vegafusion")

!gdown 1d0rPIQOhvA7q1xp5PhweNqv1FwgpVwXw
df = pd.read_csv("movies_final.csv")

df_clean = df.dropna(subset=['imdb_title_id', 'cluster'])
df.shape
```

```
Downloading...
From: https://drive.google.com/uc?id=1d0rPIQOhvA7q1xp5PhweNqv1FwgpVwXw
To: /content/movies_final.csv
100% 57.0M/57.0M [00:00<00:00, 62.6MB/s]
(84843, 30)
```

Preparación de datos y gráficos

A continuación, inicialmente preparamos ambas componentes para guardar sus valores en cada pelicula. Para ello, realizamos el mismo cálculo que en la entrega anterior y guardamos cada componente en una nueva columna.

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

class PCAPipeline(BaseEstimator, TransformerMixin):
    """
    Pipeline que aplica imputación + escalado + PCA a n componentes.
    """
    def __init__(self, n_components=2):
```

```
        self.n_components = n_components
        self.pipeline = Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
            ("pca", PCA(n_components=self.n_components))
        ])

    def fit(self, X, y=None):
        self.pipeline.fit(X)
        return self

    def transform(self, X):
        return self.pipeline.transform(X)

    def get_pca_model(self):
        return self.pipeline.named_steps["pca"]
```

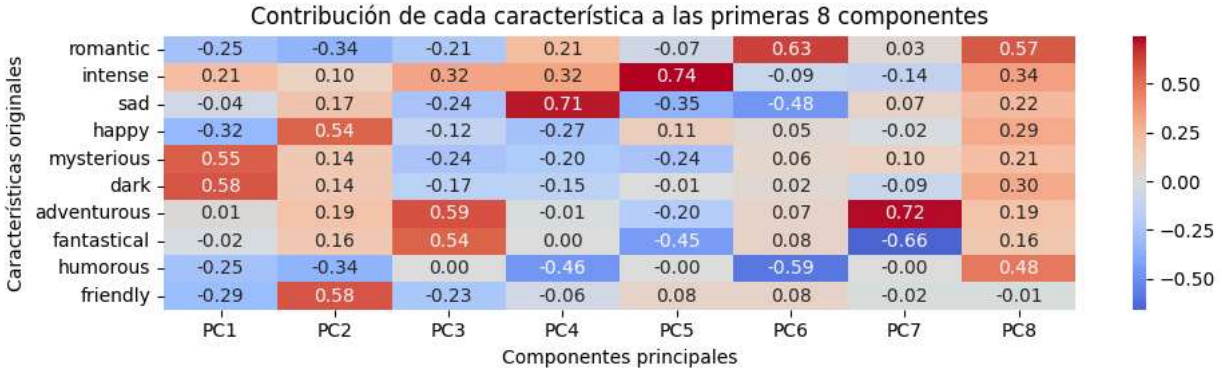
Visualización 1: Clusters y características

En esta visualización, se muestra cómo se dividen las peliculas por clusters en el gráfico de dos componentes de PCA.

Según nuestra entrega anterior, obtuvimos que al representar la información del dataset en dos componentes, estas se diferencian:

- La **componente principal 1 (PC1)** está influida positivamente por las características `dark`, `mysterious` y `happy`, pero negativamente por `humorous` y `friendly`. Como un punto de partida, aquí tendríamos películas con tonos más oscuros o de suspenso, con menor presencia de elementos ligeros o familiares.
- La **componente principal 2 (PC2)** está influida positivamente por las características `friendly` y `happy`, pero negativamente por `romantic` y `humorous`. Como un punto de partida, aquí tendríamos películas más accesibles o familiares, en contraste con otras de tono más emocional o humorístico.

En la siguiente imagen, se deja un heatmap que representa cómo contribuye cada característica a las dos componentes principales:



Además, teniendo en cuenta las dos componentes según la entrega anterior, la mejor opción era dividir las canciones en dos clusters. Revisando luego las características y géneros de las canciones contenidas en cada uno, obtuvimos que:

- El Cluster 0 está compuesto en su mayoría por peliculas ***intensas***.
- El Cluster 1 está compuesto en su mayoría por peliculas ***tranquilas***.

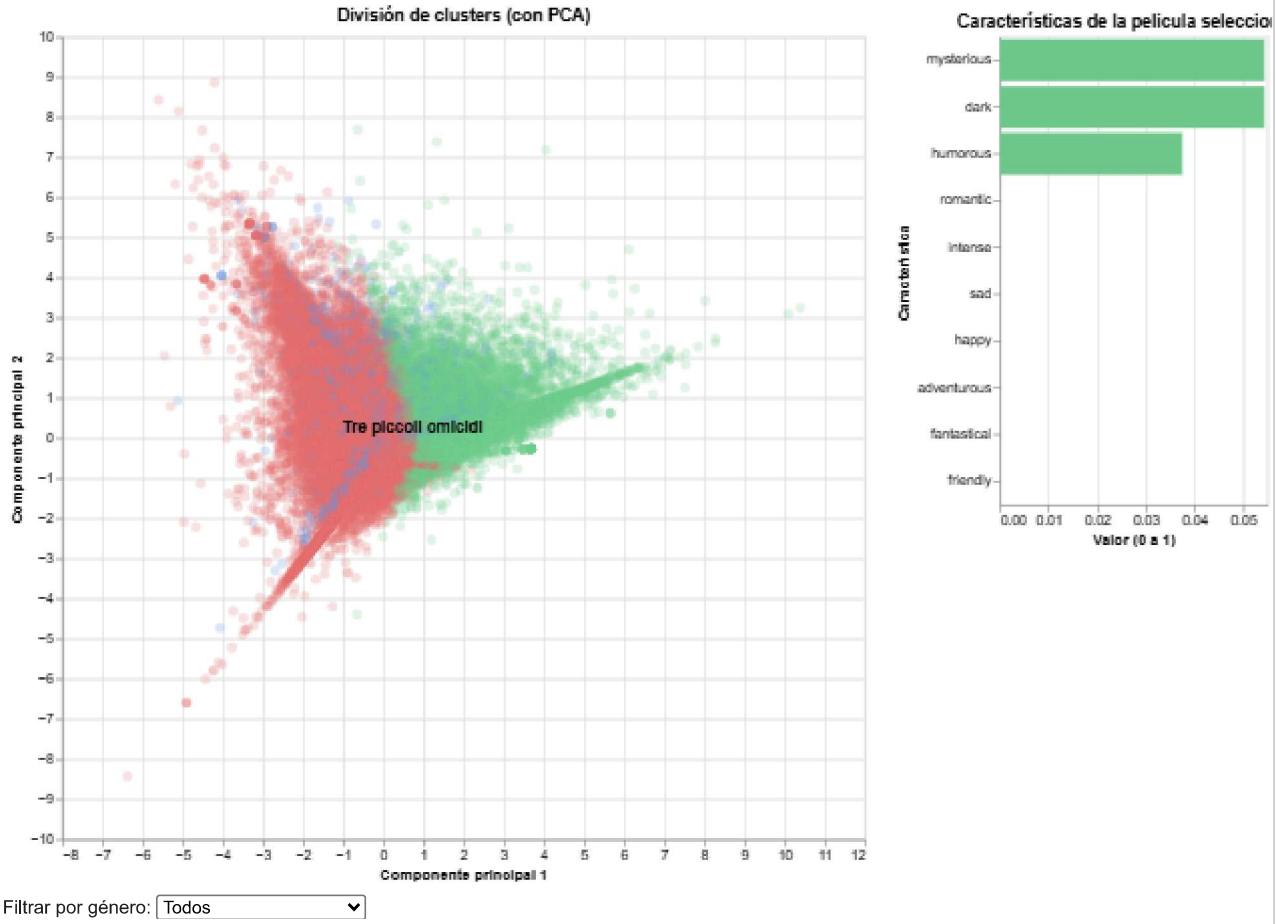
Inicialmente, se muestra como pelicula seleccionada "Giliap" de Roy Andersson. Sin embargo, el gráfico permite seleccionar otras peliculas cliqueando sobre las mismas. Al hacerlo, se mostrarán las características de cada pelicula.

```
# Seleccion de pelicula predeterminada
mbid_preferido = '85f842b8-6817-4721-a85c-8b4dde1e8814'
if mbid_preferido in df_clean['imdb_title_id'].values:
    mbid_a_seleccionar = mbid_preferido
else:
    mbid_a_seleccionar = df_clean.sample(1).iloc[0]['imdb_title_id']

initial_selection_1 = [{'imdb_title_id': mbid_a_seleccionar}]
```

```
features = ['romantic', 'intense', 'sad', 'happy', 'mysterious', 'dark','adventurous', 'fantastical', 'humorous', 'friendly'

pca2 = PCAPipeline(n_components=2)
pca_2d = pca2.fit_transform(df_clean[features])
df_clean["pca_1_2d"] = pca_2d[:,0]
df_clean["pca_2_2d"] = pca_2d[:,1]


# Características de las peliculas
features = ['romantic', 'intense', 'sad', 'happy', 'mysterious', 'dark','adventurous', 'fantastical', 'humorous', 'friendly'
```

```
# Para mostrar la canción seleccionada
#select_song = alt.selection_point(fields=["track_mbid"], on="click", clear="true")
select_song = alt.selection_point(
    fields=["imdb_title_id"],
    on="click",
    clear="true",
    value=initial_selection_1 #empty='none'
)
# Filtro por genero
genre_column = 'genre'

genres_list = df_clean[genre_column].dropna().unique().tolist()
genres_list.sort()

genre_dropdown = alt.binding_select(options=['Todos'] + genres_list, name='Filtrar por género: ')
genre_select = alt.param(bind=genre_dropdown, value='Todos')



#Colores de las referencias
color_legend = alt.Legend(
    title="Tipo de pelicula",
    labelExpr="datum.value == 0 ? 'Cluster 0 (Emocional/Romantico)' : datum.value == 1 ? 'Cluster 1 (Fantastico/Aventurero)'
)
cluster_color_scale = alt.Scale(
    domain=[0, 1, 2],
    range=['#E66E6E', '#6496E8', '#6ECC8E']
)

scatter = (
    alt.Chart(df_clean)
    .mark_circle(size=20)
    .encode(
        x=alt.X('pca_1_2d', title='Componente principal 1'),
        y=alt.Y('pca_2_2d', title='Componente principal 2'),
        color=alt.Color('cluster:N', legend=color_legend, scale=cluster_color_scale),
        opacity=alt.condition(select_song, alt.value(0.7), alt.value(0.2)),
        size=alt.condition(select_song, alt.value(150), alt.value(50)),
        tooltip=["title", "director", "genre", "cluster"],
        stroke=alt.condition(select_song, alt.value('black'), alt.value(None)),
    ).properties(
        title="División de clusters (con PCA)",
        width=600,
        height=600,
    ).add_params(
        select_song, genre_select
    )
    .transform_filter(
        alt.expr.if_(
        genre_select == 'Todos',
        True,  # no filtra nada
        alt.datum[genre_column] == genre_select  # filtra por el género seleccionado
    )
    )
    .interactive()
)


bars = (
    alt.Chart(df_clean)
    .transform_filter(select_song)
    .transform_fold(
        list(features),
        as_=["feature", "value"]
    )
    .mark_bar()
    .encode(
        x=alt.X("value:Q", title="Valor (0 a 1)"),
        y=alt.Y("feature:N", sort="-x", title="Característica"),
        color=alt.Color('cluster:N',legend=None,scale=cluster_color_scale),
        tooltip=[
            alt.Tooltip("feature:N", title="Característica"),
            alt.Tooltip("value:Q", title="Valor")
        ]
    )
    .properties(width=200, height=350, title="Características de la pelicula seleccionada")
)

# DESTACAR CANCION SELECCIONADA
text_labels = (
    alt.Chart(df_clean)
    .mark_text(
        align='center',
```

```
            baseline='bottom',
            dy=-15,
            fontSize=12,
            fontWeight='bold',
            color='black'
        )
        .encode(
            x=alt.X('pca_1_2d'),
            y=alt.Y('pca_2_2d'),
            text=alt.Text('title:N'),

            opacity=alt.condition(select_song, alt.value(1.0), alt.value(0.0))
        )
    )


final_chart = (scatter + text_labels)| bars
final_chart
```



Como se puede ver en el gráfico, se observa una clara separación visual entre los dos clusters a lo largo del eje X, el `componente principal 1 (tranquilidad)` El cluster 0 (intenso) se agrupa en el lado izquierdo, mostrando valores negativos de la componente 1. Mientras que el cluster 1 (tranquilo) se ubica del lado derecho.

Además, al seleccionar una canción se permite visualizar las características de la misma, lo que ayuda a identificar el por qué dicha canción pertenece al cluster en el que se encuentra.

Por último, el gráfico permite ver los dos clusters para cada género, ayudando a encontrar más fácilmente canciones y a distinguir características comunes entre canciones de un mismo género.

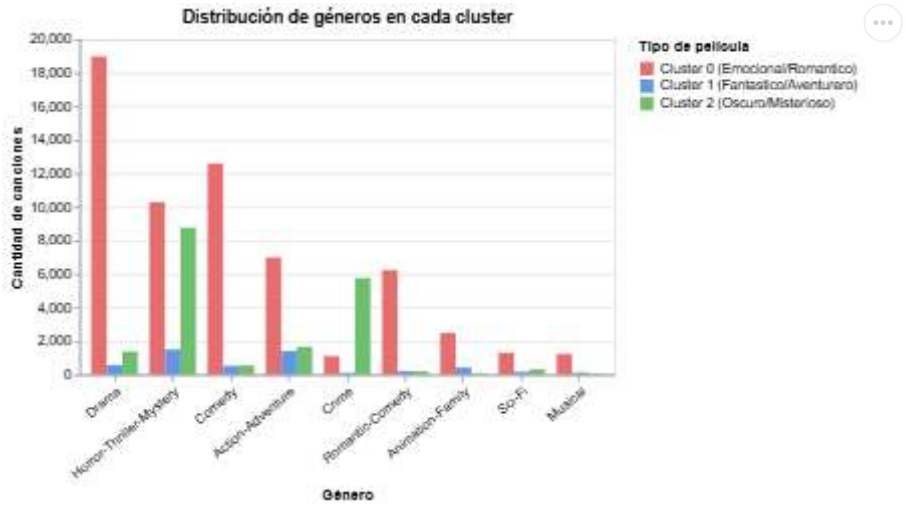
Visualización 2: Agrupación de géneros por cluster

Para validar la identidad de nuestros clusters, esta visualización nor permite vert cómo se distribuyen los géneros en los distintos clusters, permitiendo observar que hay géneros que predominan sobre un cluster.

Se utilizará un gráfico de barras donde el eje X representa los géneros musicales y el eje Y representa la cantidad total de peliculas en el dataset para ese género.

```
import altair as alt
import pandas as pd
```

```
# Contamos cuántas canciones hay por género y cluster
genre_cluster_counts = (
    df_clean.groupby(['genre', 'cluster'])
    .size()
    .reset_index(name='count')
)

# ✅ Escala de colores para 3 clusters
cluster_color_scale = alt.Scale(
    domain=[0, 1, 2],
    range=['#E66E6E', '#6496E8', '#6EBF6E']  # rojo, azul, verde
)

# Gráfico de barras agrupadas
chart = (
    alt.Chart(genre_cluster_counts)
    .mark_bar()
    .encode(
        x=alt.X('genre:N', title='Género', sort='-y', axis=alt.Axis(labelAngle=-40)),
        y=alt.Y('count:Q', title='Cantidad de canciones'),
        color=alt.Color(
            'cluster:N',
            title='Cluster',
            scale=cluster_color_scale,
            legend=alt.Legend(
                title="Tipo de película",
                # ✅ labelExpr actualizado para 3 clusters
                labelExpr="""
                    datum.value == 0 ? 'Cluster 0 (Emocional/Romantico)' :
                    datum.value == 1 ? 'Cluster 1 (Fantastico/Aventurero)' :
                    'Cluster 2 (Oscuro/Misterioso)'
                """
            )
        ),
        xOffset='cluster:N',
        tooltip=[
            alt.Tooltip('genre:N', title='Género'),
            alt.Tooltip('cluster:N', title='Cluster'),
            alt.Tooltip('count:Q', title='Cantidad de películas')
        ]
    )
    .properties(
        title='Distribución de géneros en cada cluster',
        width=400,  # un poco más ancho para acomodar 3 barras por género
        height=250
    )
)

chart
```



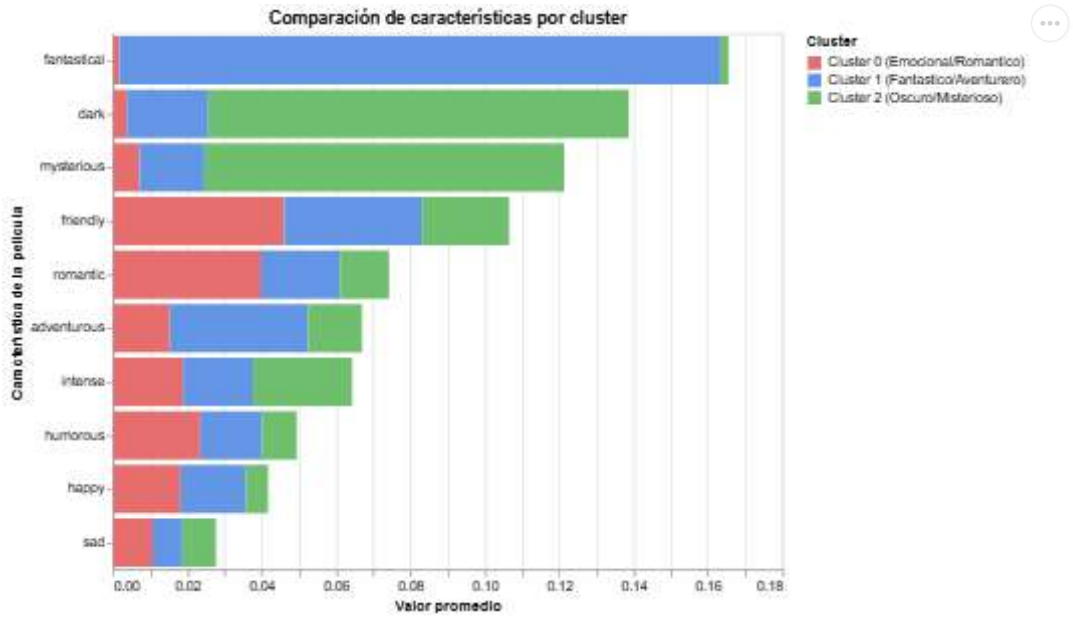
Los resultados obtenidos en este gráfico respaldan fuertemente nuestra hipótesis sobre los dos clusters y el tipo de canciones de cada uno. Ya que se puede observar que géneros más "movidos" como `Pop`, `Dance`, `Hip-Hop` y `Rock` tienen una presencia mayoritaria en el ***cluster 0 (movido)***. Mientras que géneros conocidos por su tranquilidad como `Classic` y `Jazz` pertenecen casi en su totalidad al ***cluster 1 (tranquilo)***.

Visualización 3: Comparación de características promedio por cluster

En el gráfico anterior analizamos los géneros, pero en el próximo gráfico mostraremos las características musicales que definen a cada cluster.

Realizamos un gráfico de barras horizontales agrupadas que permiten comparar el valor promedio de cada una de las cartacterísticas musicales. Para cada característica, dos barras muestran el valor promedio para cada cluster.

```
feature_means = (
    df_clean.groupby("cluster")[features].mean().reset_index().melt(id_vars="cluster")
)

# ✅ 3 clusters
cluster_color_scale = alt.Scale(domain=[0, 1, 2], range=['#E66E6E', '#6496E8', '#6EBF6E'])

chart_bar_features = (
    alt.Chart(feature_means)
    .mark_bar()
    .encode(
        y=alt.Y("variable:N", title="Característica de la pelicula", sort='-x'),
        x=alt.X("value:Q", title="Valor promedio"),
        color=alt.Color("cluster:N", title="Cluster", scale=cluster_color_scale,
                        legend=alt.Legend(
                            labelExpr="""
                                 datum.value == 0 ? 'Cluster 0 (Emocional/Romantico)' :
                                datum.value == 1 ? 'Cluster 1 (Fantastico/Aventurero)' :
                                'Cluster 2 (Oscuro/Misterioso)'
                            """
                        )),
        xOffset="cluster:N",
        tooltip=["variable:N", "cluster:N", "value:Q"]
    )
    .properties(
        title="Comparación de características por cluster",
        width=500,
        height=400
    )
)
chart_bar_features
```



Con este gráfico también podemos distinguir esa identidad de cada cluster. Esto es gracias a que se puede observar que:

- ***Cluster 0 (movido, rojo)***: tiene promedios significativamente más altos en características asociadas a la energía (`party`, `happy` y `danceable`).
- ***Cluster 1 (tranquilo)***: presenta dominancia de las características `relaxed`, `acoustic` y `sad`.

Por lo tanto, estas diferencias notables en los promedios confirman que la separación del cluster se basa en las cualidades auditivas que componen las canciones, alineandose de cierta forma con las etiquetas que definimos anteriormente "Movido" y "Tranquilo".

Visualización 4: Características de canciones anómalas

En esta última visualización, decidimos mostrar la detección de anomalías, con el objetivo de identificar las canciones que son "musicalmente atípicas" en comparación con la norma del dataset.

Entonces, utilizamos el mismo gráfico de dispersión PCA como base, pero distinguimos entre:

- Canciones normales: se muestran de color gris y con baja opacidad.
- ***Canciones anómalas: se muestran las 200 canciones más anómalas, ordenadas por su porcentaje de anomalía.***

Es importante destacar que el grado de anomalía de las canciones se codifica visualmente: a mayor anomalía mayor es el tamaño del círculo y más verde es su color (en una escala de amarillo a verde).

Al igual que con el primer gráfico, se permite seleccionar diferentes canciones para ver sus características. Inicialmente, de forma predefinida se muestra seleccionada la canción más anómala, para observar sus características.

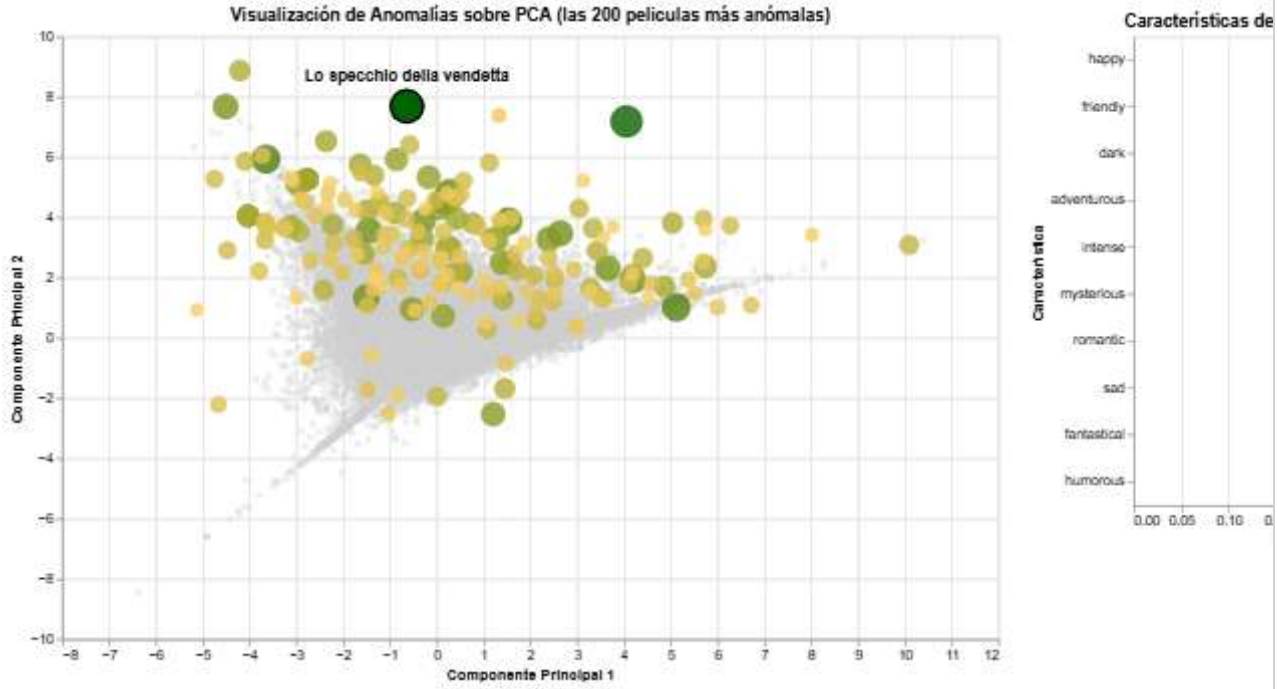
```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import MinMaxScaler

X = df_clean[features]

iso_forest = IsolationForest(contamination=0.02, random_state=42)
predictions = iso_forest.fit_predict(X)
df_clean['anomaly'] = predictions

raw_anomaly_scores = iso_forest.score_samples(X)

# Invertimos el score (multiplicando por -1) para que valores más altos sean más anómalos
inverted_scores = -raw_anomaly_scores.reshape(-1, 1)

# Usamos MinMaxScaler para escalar estos scores entre 0 (normal) y 1 (más anómalo)
scaler = MinMaxScaler()
normalized_scores = scaler.fit_transform(inverted_scores)

# Nueva columna
df_clean['porcentaje_anomalia'] = normalized_scores
```

```
# Obtener la pelicula más anomala de todas
indice_max_anomalia = df_clean['porcentaje_anomalia'].idxmax()
mbid_mas_anomalo = df_clean.loc[indice_max_anomalia]['imdb_title_id']
initial_selection = [{'imdb_title_id': mbid_mas_anomalo}]
```

```
# Escala de color
df_anomalas = (
    df_clean[df_clean['anomaly'] == -1]
    .sort_values(by='porcentaje_anomalia', ascending=False)
    .head(200)
)
min_anomalia_top200 = df_anomalas['porcentaje_anomalia'].min()
max_anomalia_top200 = df_anomalas['porcentaje_anomalia'].max()
nuevo_dominio = [min_anomalia_top200, max_anomalia_top200]

color_scale = alt.Scale(domain=nuevo_dominio, range=['#FFD166', '#006400'])
size_scale = alt.Scale(domain=nuevo_dominio, range=[100, 600])

# Selección de canción
#select_song_anomaly = alt.selection_point(fields=["track_mbid"], on="click", clear="true")
select_song_anomaly = alt.selection_point(
    fields=["imdb_title_id"],
    on="click",
    clear="true",
    value=initial_selection #empty='none'
)

# Filtro por género
genre_column = 'genre'
genres_list = df_clean[genre_column].dropna().unique().tolist()
genres_list.sort()
genre_dropdown = alt.binding_select(options=['Todos'] + genres_list, name='Filtrar por género: ')
genre_select = alt.param(bind=genre_dropdown, value='Todos')

# Escala para cluster (la que usás en el gráfico de barras)
cluster_color_scale = alt.Scale(domain=[0, 1], range=['#E66E6E', '#6496E8'])

# --- CAPA 1: Canciones normales ---
normales = alt.Chart(df_clean[df_clean['anomaly'] == 1]).mark_circle(size=25).encode(
    x=alt.X('pca_1_2d', title='Componente Principal 1'),
    y=alt.Y('pca_2_2d', title='Componente Principal 2'),
    color=alt.value('#D3D3D3'),
    opacity=alt.value(0.3),
    stroke=alt.condition(select_song_anomaly, alt.value('black'), alt.value(None)),
    strokeWidth=alt.condition(select_song_anomaly, alt.value(2), alt.value(0))
).transform_filter(
    alt.expr.if_(
```

```
        genre_select == 'Todos',
        True,
        alt.datum[genre_column] == genre_select
    )
)

# --- CAPA 2: Canciones anómalas ---
anomalas = alt.Chart(
    df_clean[df_clean['anomaly'] == -1]
    .sort_values(by='porcentaje_anomalia', ascending=False)
    .head(200)
).mark_circle().encode(
    x='pca_1_2d',
    y='pca_2_2d',

    fill=alt.Fill(
        'porcentaje_anomalia:Q',
        scale=color_scale,
        legend=None
    ),

    size=alt.Size(
        'porcentaje_anomalia:Q',
        scale=size_scale,
        legend=alt.Legend(
            title='Nivel de Anomalía',
            format='.0%',
            orient='right',
            titleFontSize=13,
            labelFontSize=11,
        )
    ),
    tooltip=[
        'title', 'director', 'genre',
        alt.Tooltip('porcentaje_anomalia:Q', format='.2%')
    ],
    opacity=alt.condition(select_song_anomaly, alt.value(1), alt.value(0.8)),
    stroke=alt.condition(select_song_anomaly, alt.value('black'), alt.value(None)),
    strokeWidth=alt.condition(select_song_anomaly, alt.value(2), alt.value(0))
).transform_filter(
    alt.expr.if_(
        genre_select == 'Todos',
        True,
        alt.datum[genre_column] == genre_select
    )
)


text_labels = (
    alt.Chart(df_clean)
    .mark_text(
        align='center',
        baseline='bottom',
        dy=-15,
        fontSize=12,
        fontWeight='bold',
        color='black'
    )
    .encode(
        x=alt.X('pca_1_2d'),
        y=alt.Y('pca_2_2d'),
        text=alt.Text('title:N')
    )
    .transform_filter(select_song_anomaly)
)


# --- GRÁFICO PRINCIPAL CON LAS DOS CAPAS ---
scatter_anomalies = alt.layer(normales, anomalas, text_labels).properties(
    title='Visualización de Anomalías sobre PCA (las 200 peliculas más anómalas)',
    width=700,
    height=450
).add_params(
    genre_select,
    select_song_anomaly
).interactive()

# --- GRÁFICO DE BARRAS CON CARACTERÍSTICAS ---
bars = (
    alt.Chart(df_clean)
    .transform_filter(select_song_anomaly)
    .transform_fold(
```

```
            list(features),
            as_=["feature", "value"]
        )
        .mark_bar()
        .encode(
            x=alt.X("value:Q", title="Valor"),
            y=alt.Y("feature:N", sort="-x", title="Característica"),
            color=alt.Color('cluster:N', legend=None, scale=cluster_color_scale),
            tooltip=[
                alt.Tooltip("feature:N", title="Característica"),
                alt.Tooltip("value:Q", title="Valor")
            ]
        )
        .properties(width=250, height=350, title="Características de la pelicula seleccionada")
    )

    # --- COMBINAR EN UN SOLO LAYOUT ---
    final_chart = scatter_anomalies | bars
    final_chart
```



Filtrar por género: Todos

El gráfico revela que las canciones anómalas no se concentran en una sola región, sino que aparecen distribuidas.

Al cargar, el gráfico selecciona automáticamente la canción ***más anómala***, "Trace", que tiene un porcentaje_anomalia del 100%. En el gráfico de barras de la derecha podemos analizar por qué es atípica. Y observando esta canción (la más atípica), podemos entender el por qué de esta clasificación: posee altos porcentajes de características que son totalmente opuestas, como `happy` y `sad`.